Tracing Recursion

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **trace** a recursive method's calls on the call stack.
2. Be able to **classify** a recursive method's Big-O by its calls and the work in each.

Recap: factorial

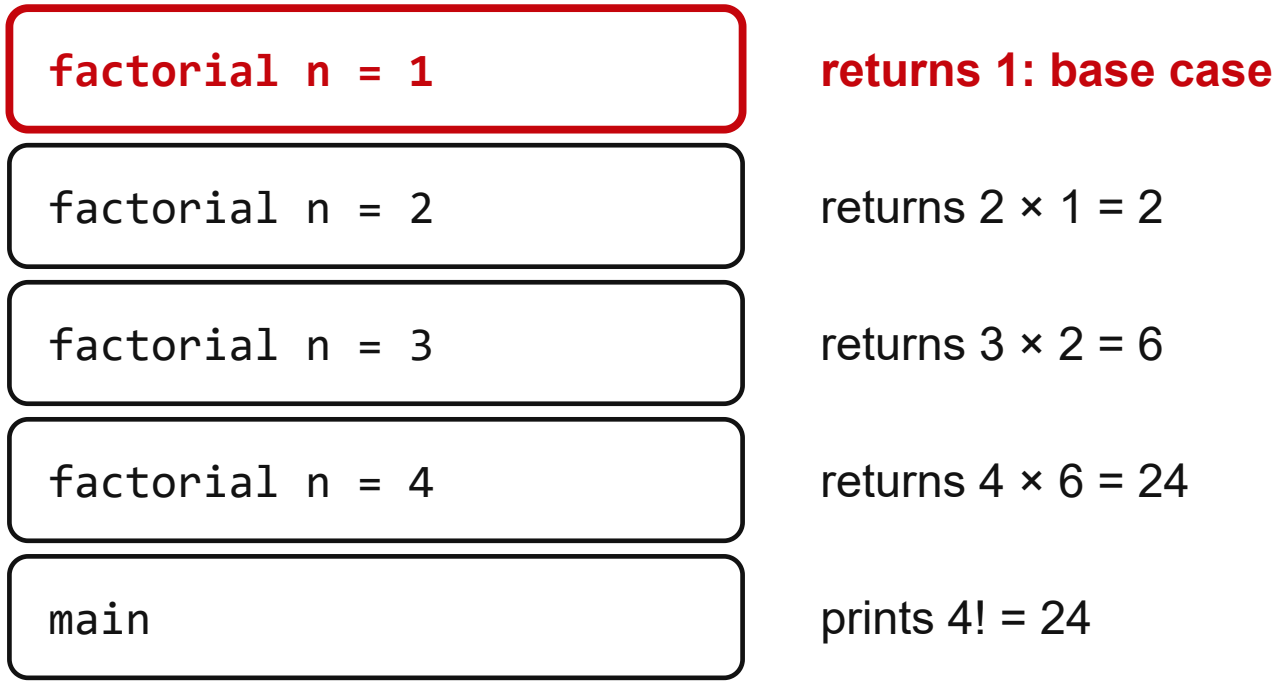
```
public static int factorial(int n) {
  if (n == 1) {
    return 1;                    // base case: 1! is 1
  }
  return n * factorial(n - 1);   // recursive case: one smaller
}
```

No base case? Or no progress toward it?

`StackOverflowError` on `n <= 0`.

The call stack

Reminder: every call gets its own **frame** on the stack, with its own `n`.



Watch the stack

Each call **waits** for the call it made. At the base case, the frames start to pop, and each one does its multiply on the way out.

→ Step through `factorial(4)` in a visualization.

Tophat Q1

What does `f(3)` *return*?

```
public static int f(int n) {
  if (n <= 0) {
    return 0;
  } else {
    int result = f(n - 1);
    System.out.print(n + " ");
    return 1 + result;
  }
}
```

→ Open Tophat to vote.

Tophat Q2

What does `f(3)` *print*?

```
public static int f(int n) {
  if (n <= 0) {
    return 0;
  } else {
    int result = f(n - 1);
    System.out.print(n + " ");
    return 1 + result;
  }
}
```

→ Open Tophat to vote.

Tophat Qs

Let's walk through them together!

```
public static int f(int n) {
  if (n <= 0) {
    return 0;
  } else {
    int result = f(n - 1);
    System.out.print(n + " ");
    return 1 + result;
  }
}
```

Exercise: `longest`

Same method as last lecture, re-visiting in detail.

Big-O of a recursive method

1. How much **work** does each call do?
2. How many **calls** are there?

Multiply them. With `longest` ...

- Original method: `O(N)`
- Copy every call: `O(N * N) = O(N²)`
- Copy once, use throughout: `O(N + N) = O(N)`

... or is it?

A hidden loop

N calls. But `remove(0)` shifts every song after it down one spot: **a loop, hidden in a method call**.

N calls × up to N shifts each: N + (N – 1) + … + 1, about $N^2/2$. **$O(N^2)$**.

… all of the complexities on the previous slide? Multiply them by an additional **N**! 😲 Can we do better?

Remove from the end

```
int last = songs.size() - 1;
if (songs.get(last).getSeconds() < songs.get(last - 1).getSeconds()) {
  songs.remove(last);              // nothing after it to shift
} else {
  songs.remove(last - 1);          // shifts just 1 song
}
```

N calls × O(1) work each: **O(N)**. We have to look at every song once anyway, so we can't beat O(N).

Exercise: `longest`

Create the optimal, recursive `longest`.

Tophat Q3

In the following function, what is the base case(s), and what is the recursive case(s)?

```
public static int g(String word, int i) {
  if (i == word.length()) {
    return 0;
  } else if (word.charAt(i) == 'x') {
    return g(word, i + 1) + 1;
  } else {
    return g(word, i + 1) + 0;
  }
}
```

Tophat Q4

In the following function, what is the result of `g("xxhixx", 1)`?

```
public static int g(String word, int i) {
  if (i == word.length()) {
    return 0;
  } else if (word.charAt(i) == 'x') {
    return g(word, i + 1) + 1;
  } else {
    return g(word, i + 1) + 0;
  }
}
```

Tophat Qs

Let's walk through them together!

```
public static int g(String word, int i) {
  if (i == word.length()) {
    return 0;
  } else if (word.charAt(i) == 'x') {
    return g(word, i + 1) + 1;
  } else {
    return g(word, i + 1) + 0;
  }
}
```

Tophat Q5

Suppose there exists a function `h` ...

```
public static int h(String word) {
  return g(word, 0);
}
```

What is the Big O of `h`, where `N` is the length of word?

→ Open Tophat to vote.

Tophat Q6

Suppose there exists a function `h` ...

```
public static int h(String word) {
  return g(word, 0);
}
```

What is Ω? Is there a Θ?

→ Open Tophat to vote.

Questions?